

Social Support AI

Eligibility Assessment Prototype

Solution Summary & Technical Design Document

Prepared for: XDigit / Department of Government Enablement (DGE), Abu Dhabi
Prepared by: Muhammad Haris

1. Executive Summary

This document presents a prototype AI system that automates the intake, validation, and eligibility assessment process for government social/financial support applications. The current manual process takes 5 to 20 working days per application. This prototype demonstrates an end-to-end automated pipeline capable of reducing that to a near-instant, AI-assisted decision, while maintaining transparency through explainable reasoning and human-reviewable validation flags.

The system ingests five applicant documents (bank statement, credit report, Emirates ID, an assets/liabilities spreadsheet, and an optional resume), extracts and cross-validates the relevant data, applies a trained machine learning model to assess eligibility, and uses a locally-hosted large language model to generate a plain-language, fact-checked explanation of the decision along with a relevant enablement recommendation (e.g. job training, career counseling).

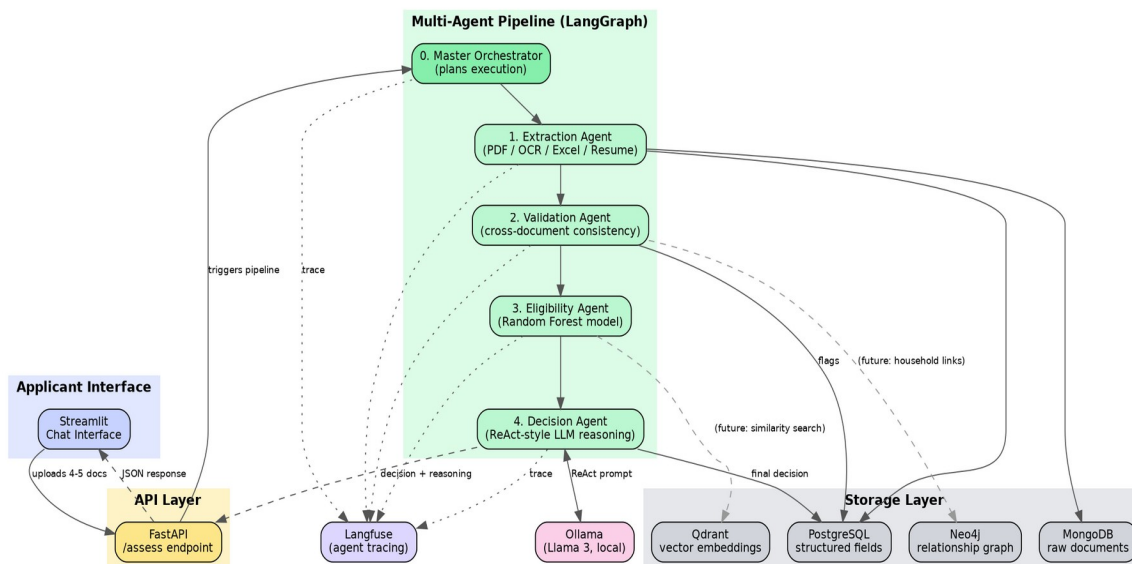
2. Problem Statement

Government social support departments currently process applications through a largely manual review workflow. Caseworkers must individually review submitted documents, verify financial details, cross-check consistency, and make an eligibility determination. This is time-consuming, inconsistent across reviewers, and creates a poor experience for applicants who may be in genuine financial need.

The goal of this prototype is to demonstrate how AI can automate the repetitive, data-extraction-heavy parts of this process while keeping the system transparent and auditable, so that human caseworkers remain in control of final decisions for edge cases.

3. Architecture Overview

The system is composed of five layers: an applicant-facing chat interface, an API layer, a multi-agent processing pipeline, a local LLM for reasoning, and a multi-database storage layer. The diagram below shows how these components interact.



Solid arrows represent data flow implemented and demonstrated in this prototype. Dashed arrows represent connections that are architecturally in place (the databases are provisioned and reachable) but not yet fully exercised by the current pipeline logic — these are called out explicitly in Section 8 (Limitations & Future Improvements).

4. Workflow Breakdown

Step 1 — Document Upload

The applicant uploads their documents through the Streamlit chat interface: a bank statement (PDF), a credit report (PDF), an Emirates ID (image), an assets/liabilities spreadsheet (Excel), and an optional resume (PDF) used to derive employment history and family size.

Step 2 — Master Orchestrator Agent

Before any document is processed, the orchestrator agent inspects the submission, confirms which required documents are present, checks whether a resume was provided, and builds an explicit execution plan. This plan is logged and traced, and determines whether the pipeline can proceed or should halt due to missing required documents.

Step 3 — Extraction Agent

The extraction agent parses each document type using a dedicated method: pdfplumber for layout-aware PDF text and table extraction, pandas/openpyxl for the Excel spreadsheet, Tesseract OCR for the ID image, and a resume parser for employment status, years of employment, and family size. Extracted fields are combined into a single structured applicant record.

Step 4 — Validation Agent

The validation agent cross-checks the combined record for consistency: does the applicant's name match across all documents? Does the Emirates ID match between the ID card and credit report? Are critical fields (income, credit score) missing? Are financial ratios plausible? Any issues are recorded as validation flags with a severity level, rather than blocking the pipeline outright.

Step 5 — Eligibility Agent

The eligibility agent applies a trained Random Forest classifier to the applicant's financial features (income per capita, debt-to-asset ratio, employment status, credit score, years of employment, family size) and returns a decision with a confidence score. Decisions of 'Declined' with confidence below a 65% threshold are automatically downgraded to 'Soft Decline' — signalling a borderline case recommended for human caseworker review rather than an automatic rejection.

Step 6 — Decision Agent

The decision agent uses a structured, ReAct-inspired prompt (Thought -> Action -> Observation -> Final Answer) that asks the locally-hosted Llama 3 model, via Ollama, to explicitly reason through the applicant's figures before committing to an explanation. This is a lightweight, single-pass adaptation of the ReAct pattern rather than a full multi-turn tool-calling agent loop — the intermediate reasoning steps are still captured and stored for auditability. A post-generation consistency check compares the LLM's explanation against the applicant's actual figures (e.g. flags an explanation that calls a high income 'low') and substitutes a guaranteed-accurate rule-based explanation if a contradiction is detected. If the local LLM is unreachable entirely, the same rule-based fallback is used.

Step 7 — Orchestration & Observability

All five agent steps (orchestrator, extraction, validation, eligibility, decision) are wired together using LangGraph as an explicit state graph, making the pipeline's flow easy to visualize, debug, and extend with additional steps or conditional branches. Every step's inputs, outputs, duration, and errors are traced via Langfuse when configured, or to a structured local log otherwise — giving full observability into each agent's behavior without requiring a cloud account for local development.

Step 8 — Response

The final decision, confidence score, financial summary, employment details, reasoning, and any validation flags are returned to the Streamlit interface and displayed to the applicant in a clear, color-coded summary (green for Approved, amber for Soft Decline, red for Declined).

5. Technology Stack & Tool Justifications

Tools were selected against five dimensions specified in the assignment brief: suitability to the problem, scalability, maintainability, performance, and security.

Backend & Frontend

- FastAPI — Suitability: async-native, automatic OpenAPI docs speed up integration. Scalability: async workers handle concurrent requests well; horizontally scalable behind a load balancer. Maintainability: type-hinted, self-documenting via Pydantic models. Performance: one of the fastest Python web frameworks available. Security: built-in request validation reduces injection risk; would need auth middleware added for production.
- Streamlit — Suitability: fastest path to an interactive chat-style demo UI without a separate frontend build. Scalability: adequate for prototype/low-concurrency use; not designed for high-traffic production frontends. Maintainability: single-file Python, no build pipeline. Performance: acceptable for demo purposes; full page reruns can be a bottleneck at scale. Security: no built-in auth — would need to sit behind a proper identity layer in production.

Data Layer

- PostgreSQL — Suitability: strong fit for structured, relational applicant and financial records. Scalability: proven at large scale with read replicas and partitioning. Maintainability: mature tooling, well-understood operationally. Performance: strong for transactional structured queries. Security: mature row-level security and encryption support.
- MongoDB — Suitability: flexible schema fits variably-shaped raw extracted document content. Scalability: horizontal sharding support. Maintainability: schema flexibility reduces migration overhead as document types evolve. Performance: fast writes for document-shaped data. Security: field-level encryption and RBAC available.
- Qdrant — Suitability: purpose-built vector similarity search for future case-matching features. Scalability: designed for horizontal scaling of vector search. Maintainability: simple API, self-hostable. Performance: optimized ANN search. Security: self-hosted keeps data on-premises, important for government data residency requirements.
- Neo4j — Suitability: natural fit for household/family relationship modeling. Scalability: causal clustering for larger deployments. Maintainability: Cypher query language is expressive and readable. Performance: efficient for relationship-heavy queries relational databases handle poorly. Security: role-based access control available.

AI / ML Layer

- scikit-learn (Random Forest) — Suitability: strong baseline for tabular classification with mixed numeric/categorical features, and the dataset size (hundreds of rows) doesn't yet justify a deep learning approach. Scalability: fast training/inference; would need retraining pipelines at larger data volumes. Maintainability: simple to retrain, version, and explain to non-technical stakeholders. Performance: millisecond-level inference. Security: model artifacts are local files requiring standard access control, no external API exposure of applicant data.
- Ollama + Llama 3 (local LLM) — Suitability: keeps sensitive applicant data on-premises rather than sent to a third-party API. Scalability: current single-instance local hosting does not scale to concurrent production load; would need GPU-backed serving infrastructure. Maintainability: simple model swapping via Ollama's model registry. Performance: 30-90+ seconds per call observed on non-GPU hardware — a real constraint

noted in Section 8. Security: no data leaves the local environment, which is significant for government data sensitivity.

- LangGraph — Suitability: explicit state-graph model matches the multi-step agent pipeline well. Scalability: stateless graph execution can be parallelized across requests. Maintainability: adding new pipeline steps is a graph edge, not a rewrite. Performance: negligible orchestration overhead compared to the LLM/model calls themselves. Security: no external calls introduced beyond what individual nodes already make.
- Docker Compose — Suitability: reproducible multi-database local environment. Scalability: intended for local development; production would move to managed database services or Kubernetes. Maintainability: single command environment setup. Performance: no meaningful overhead for a prototype. Security: default credentials used in this prototype must be replaced before any real deployment.

6. Data & Model Details

Given the absence of real historical application data, a synthetic dataset of 300 applicant profiles was generated with realistic distributions for income, family size, employment status, liabilities, assets, and credit score. Eligibility labels were assigned using a rule-based function approximating plausible real-world criteria (e.g. low income-per-capita and high debt-to-asset ratio increase approval likelihood; high assets and high income-per-capita decrease it), with 5% label noise introduced to avoid an unrealistically clean decision boundary.

A Random Forest classifier (200 trees, max depth 8) was trained on an 80/20 train-test split, achieving 90% accuracy on held-out data. Three representative test cases (a low-income applicant, a high-income applicant, and a borderline mid-income applicant) were used to validate that the full document-to-decision pipeline produces sensible, differentiated outcomes across Approved, Declined, and Soft Decline categories. Employment history and family size used by the model are extracted from each applicant's resume rather than assumed, closing an earlier prototype limitation where these fields were hardcoded defaults.

7. Validation & Error Handling

- Name and Emirates ID consistency checks across all uploaded documents
- Missing critical field detection (income, credit score)
- Plausibility checks on debt-to-income and credit score ranges
- Soft Decline category: low-confidence rejections are routed to human review rather than treated as a definitive automated no
- LLM reasoning consistency check: generated explanations are compared against the applicant's actual figures, and discarded in favor of a guaranteed-accurate rule-based explanation if a contradiction is detected (e.g. describing a high income as 'low')
- Graceful degradation: if the local LLM is unreachable, the system falls back to rule-based reasoning rather than failing the request
- All validation flags and orchestrator planning notes are persisted and traced via Langfuse (or a local structured log) rather than silently discarded

8. Limitations & Future Improvements

This prototype demonstrates the core architecture and decision pipeline, and closes several gaps identified during development (resume ingestion, real employment/family data, an explicit orchestrator agent, observability tracing, and reasoning-accuracy safeguards). Several areas would still need further development for production readiness:

- Document format flexibility: extraction logic is currently matched to a specific synthetic document layout. A production system would need LLM-based flexible extraction or a library of per-institution templates to handle real-world document variability.

- Human-in-the-loop review: Soft Decline cases are flagged for review, but there is no actual caseworker review dashboard or workflow yet — this is the natural next feature.
- Model training data: the current model is trained on synthetic data; production use requires training on real (anonymized) historical decisions with proper validation against known outcomes.
- Vector and graph database utilization: Qdrant and Neo4j are provisioned and architecturally integrated but not yet exercised by the current pipeline logic. Planned use includes semantic similarity search across past cases (Qdrant) and household/family relationship modeling for fraud and duplicate-application detection (Neo4j).
- Reasoning framework depth: the decision agent uses a single-pass ReAct-style prompt rather than a full multi-turn tool-calling agent loop (e.g. one that could query the database mid-reasoning or re-check a flagged field). This was a deliberate scope decision for a time-boxed prototype; a production system would benefit from a true agentic loop with tool access.
- Explainability: adding SHAP value analysis would let caseworkers see exactly which features drove each model decision, beyond the LLM's natural-language summary.
- Security & compliance: production deployment requires encryption at rest and in transit, role-based access control, full audit logging, and compliance review against UAE data protection regulations.
- Performance: local LLM inference latency (30-90+ seconds observed on CPU-only hardware) would need dedicated GPU infrastructure for acceptable production response times at scale.

Integration into Existing Government Systems

This prototype is designed with integration in mind rather than as a standalone silo:

- API-first design: the FastAPI backend exposes a clean /assess endpoint independent of the Streamlit frontend, so it could be called directly by an existing case-management system, a government portal, or a caseworker's internal tooling without any changes to the pipeline itself.
- Data pipeline considerations: the extraction layer is decoupled from storage, so a production integration could swap the current synthetic-document parsers for connectors to existing document management systems, national ID verification services, or bank data-sharing APIs (e.g. Open Banking-style integrations) with the rest of the pipeline unchanged.
- Incremental adoption path: the system could initially run in a shadow/advisory mode alongside existing manual review — surfacing its decision and reasoning to caseworkers as a recommendation rather than an automated action — before being trusted with automated decisions for low-risk, high-confidence cases only.
- Audit trail: every decision, validation flag, and orchestrator plan is persisted to PostgreSQL and traced via Langfuse, giving any integrating system a queryable history of how and why each decision was made.

9. Conclusion

This prototype demonstrates a complete, working, end-to-end AI-assisted eligibility assessment pipeline: a master orchestrator plans execution, extraction agents process five document types (including resume-derived employment data), a validation agent cross-checks consistency, a machine learning model assesses eligibility with a Soft Decline safety category for borderline cases, and a decision agent generates plain-language, fact-checked reasoning using a locally-hosted LLM — all wrapped in an interactive chat interface with full observability tracing. While built with synthetic data and with several honestly-documented limitations, the architecture is designed to be extended into a production-grade system with the enhancements outlined in Section 8.